

RBAC Fundamentals: Who Can Do What, Where?

Master Azure's Role-Based Access Control — the permission system behind every resource

 Beginner

 15 Minutes

 AZ-104 · Module 01

Why This Matters

Every Azure action requires permission. Without understanding RBAC (Role-Based Access Control), you might accidentally give a junior developer the ability to delete your entire production environment—or prevent them from doing their job. RBAC is Azure's permission system that keeps your resources secure.

Before You Start

Prerequisites	00-Prerequisites: Identity & Access Fundamentals
Time to understand	15 minutes
Difficulty	 Beginner (foundational, no experience needed)
What you'll learn	What RBAC is, how it works, why it matters

The Simple Idea

What Is RBAC?

RBAC = Role-Based Access Control. It answers one question: **"Who can do what, where?"**

A Hotel Key Card Analogy

Imagine a hotel:

- **A guest** gets a key card that opens only their room
- **A manager** gets a card that opens all guest rooms AND the office
- **A janitor** gets access to cleaning closets only

Nobody gets a "master key to everything." Everyone gets exactly what they need.

In Azure, the same idea:

- A **role** = permission level (like a key card)
- A **user** = person using the card (like a hotel guest)
- A **scope** = where the card works (guest rooms, office, etc.)

Hotel

|

├ Guest → key card opens only their room

|

├ Manager → key card opens all guest rooms + office

|

└ Janitor → key card opens cleaning closets only

Azure equivalent

|

├ Role = permission level (the key card)

├ User = person using it (the guest)

└ Scope = where it works (guest rooms, office, etc.)

Nobody gets a "master key to everything."

The Three RBAC Questions

Every permission in Azure answers these three questions:

Question	Answer	Example
WHO?	Which user/service needs access?	Sara.Johnson@company.com

6/15/26, 1:28 PM

RBAC Fundamentals - Who Can Do What, Where?

WHAT?	What can they do?	Create and modify VMs (but not delete)
WHERE?	On which resource/scope?	Only the "Development" resource group

How It Actually Works

The 3-Step Permission Flow

Step 1: Identity exists in Entra ID

Sara Johnson creates a user account in Entra ID

↓

Now Azure knows "who" Sara is

Step 2: Assign a role

Admin assigns Sara the role:
"Virtual Machine Contributor"

↓

This role includes permission to:
create VMs, start VMs, modify VMs

Step 3: Scope the assignment

Admin limits this role to the
"Dev" resource group only

↓

Result: Sara can create VMs in "Dev"
but NOT in "Production"

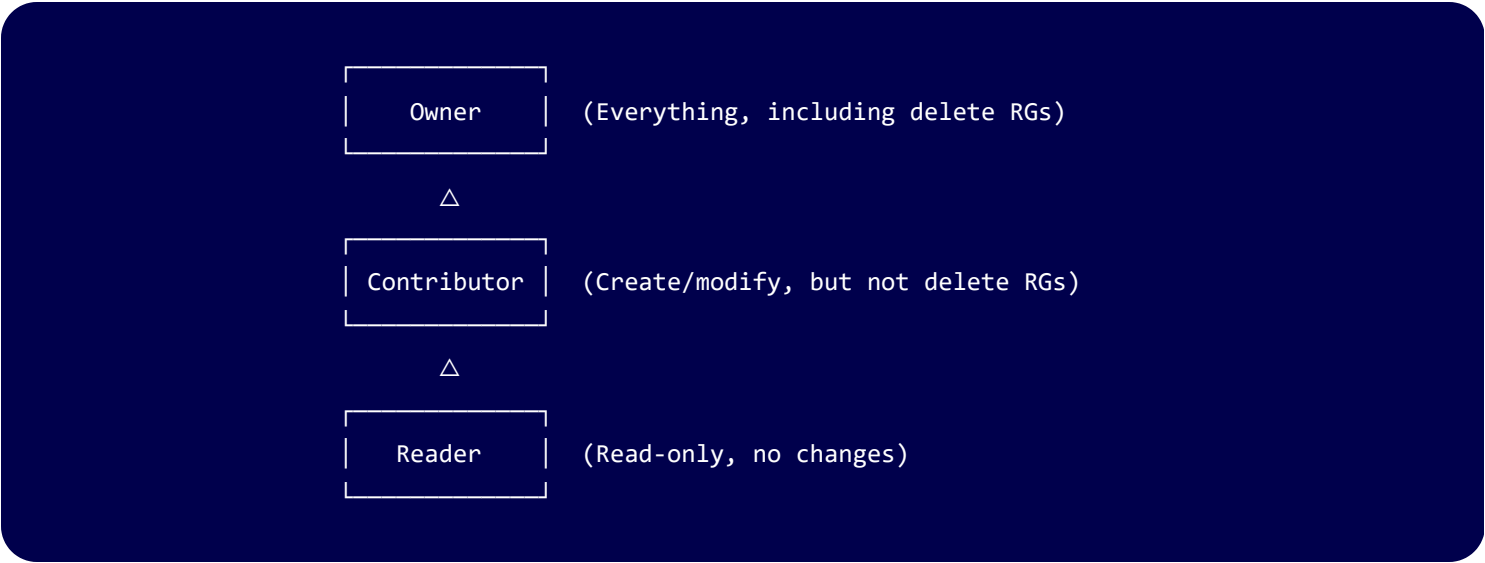
Key RBAC Components

Component	What It Does	Example
-----------	--------------	---------

Identity	The "who" — person or service	User: sara@company.com, Service: Data Factory
Role	The "what" — list of allowed actions	Contributor, Reader, Custom Role
Scope	The "where" — which resources apply to	Resource Group, Subscription, Single Resource
Assignment	The connection tying them together	Sara = Contributor on "Dev" RG

Mental Model: The Permission Pyramid

Think of Azure permissions like a pyramid:



Each role is more limited than the one above. Least privilege = more security.

Worked Example: Real Scenario

The Scenario

Situation: You're the Azure Admin at TechCorp. You hired three people:

- **Sara:** Junior Developer (needs to deploy VMs in Dev)

- **Mike:** Senior Developer (needs everything Sara has + access to Production)
- **Bob:** Database Admin (only needs SQL Server access)

Your goal: Give each person **exactly** what they need, nothing more.

Step-by-Step Solution

Sara's Setup (Least Privilege)

1. Create user: sara@techcorp.com in Entra ID
2. Assign role: "Virtual Machine Contributor"
(allows: create, modify, start/stop VMs)
3. Scope: "Dev" resource group ONLY
4. Result: Sara can work on Dev VMs,
can't touch Production

Why this role? She needs to create/modify VMs but NOT delete resource groups.

Mike's Setup (More Permissions)

1. Create user: mike@techcorp.com in Entra ID
2. Assign role: "Contributor"
(allows: create, modify, delete
everything EXCEPT RGs)
3. Scope: Both "Dev" AND "Production"
resource groups
4. Result: Mike has full control in
both Dev and Production

Why this role? Senior dev needs more access across environments.

Bob's Setup (Specific to Service)

1. Create user: bob@techcorp.com in Entra ID
2. Assign role: "SQL Server Contributor"
(allows: only SQL-related actions)
3. Scope: "Production" SQL Server
resource only

4. Result: Bob can manage SQL, can't see VMs or other resources

Why this role? Database admin doesn't need VM access—just SQL.

Assignment Table

User	Role	Scope	Can Do	Cannot Do
Sara	Virtual Machine Contributor	Dev RG	Create/modify VMs	Delete anything, touch Production
Mike	Contributor	Dev + Prod RGs	Create/modify/delete resources	Delete resource groups
Bob	SQL Server Contributor	Prod SQL Server	Manage SQL database	Access VMs, change network settings

Common Mistakes (What NOT to Do)

✗ Mistake 1: Everyone Gets Owner Role

Wrong

Admin assigns Owner role to all 50 team members
↓
Any person can delete the entire subscription accidentally
↓
Disaster: Entire production environment deleted

Fix

- Use least privilege:
- Give most people: Contributor or specific roles
 - Give only senior admins: Owner role
 - Review periodically to remove unnecessary permissions

Why it fails: Owner = complete access including destructive actions. One mistake = catastrophe.

✗ Mistake 2: Assignment at Subscription Level for Developers

Wrong

```
Dev gets Contributor role at Subscription level
↓
Dev can access ALL resource groups (Dev, QA, Production)
↓
Dev accidentally modifies Production database
```

Fix

```
Scope role assignments to the minimum needed:
• Dev → Assign to "Dev" resource group only,
  NOT subscription
• Limits blast radius if credentials are compromised
• Prevents accidental access to production
```

Why it fails: Subscription-level scope = access to everything.

✗ Mistake 3: Creating Custom Roles for Everything

Wrong

```
Admin creates 20 custom roles instead of using built-in ones
↓
Difficult to maintain and audit
↓
Inconsistent permissions across teams
```

Fix

```
Start with built-in roles:
• Use "Virtual Machine Contributor" instead of
  custom VM role
• Use "SQL Server Contributor" for database work
```

- Only create custom roles when no built-in role fits (rare)

Why it fails: Complexity = security gaps. Built-in roles are tested and documented.

✗ Mistake 4: Permanent High-Privilege Access

Wrong

```
Junior dev gets Owner role "temporarily"
↓
6 months later, still has Owner role
(nobody revoked it)
↓
Security breach: Compromised credentials
= full access
```

Fix

```
Use Privileged Identity Management (PIM):
• Users request temporary access when needed
• Auto-revoke after time limit expires
• Full audit trail of who accessed what and when
```

Why it fails: "Temporary" becomes permanent. Accounts grow more permissions over time.

Summary Table: RBAC Roles at a Glance

Role	Permissions	Best For	Caution
Owner	Everything including delete RGs	Azure admins only	Too much power—restrict access
Contributor	Create/modify/delete resources (not RGs)	Senior devs, operators	Not for juniors
Reader	Read-only, no changes	Auditors, viewers	Good for "look but don't touch"

Specific roles	Limited to one service (VM Contributor, SQL Contributor)	Developers, DBAs	Recommended—least privilege
Custom roles	User-defined permissions	When no built-in role fits	Rare; use built-ins first

How This Connects to Other Topics

Related to Module 02 (Storage)

- **Private Endpoints:** Combined with RBAC, they provide end-to-end security for storage
- **Storage Access:** Use RBAC roles like "Storage Blob Data Contributor" to control who accesses storage

Related to Module 03 (Compute)

- **VM Access:** Use RBAC to control who can create/delete VMs
- **Managed Identities:** VMs use RBAC to access other resources securely

Related to Module 04 (Networking)

- **NSGs vs RBAC:** NSGs control *network* access (which IP can reach which port)
- **RBAC** controls *Azure service* access (who can modify network settings)
- **Combined:** NSG (network boundary) + RBAC (who controls it) = complete security

Related to Module 05 (Monitor)

- **Audit logs:** See who made changes and when (linked to RBAC assignments)
- **Diagnostics:** Only certain roles can enable monitoring

See It In Action

Associated lab: Lab 02: RBAC & Azure Policy

Suggested learning sequence:

1.  Read this doc (RBAC Fundamentals)

2.  Read **Entra ID Overview** (identities come first)
3.  Read **Management Groups & Policy** (policy enforces RBAC)
4.  Work through Lab 02 (hands-on assignment)
5.  Read **Access Control Scenarios** (real-world patterns)

Key Takeaways

-  **Summary**
- **RBAC = WHO + WHAT + WHERE:** The three components of every permission
 - **Least Privilege:** Give people only the minimum access they need
 - **Scope Matters:** Resource group-level access is better than subscription-level
 - **Built-in Roles First:** Use Azure's tested roles before creating custom ones
 - **Review Regularly:** Remove unused permissions to stay secure
 - **RBAC ≠ NSG:** RBAC controls who uses Azure services; NSGs control network traffic

Next Steps

1. **Read first: Entra ID Overview** (understand identities)
2. **Then read: Management Groups & Policy** (policy + RBAC together)
3. **Practice: Lab 02: RBAC & Azure Policy**
4. **Apply: Access Control Scenarios** (patterns for your org)

Azure AZ-104	Module	Generated
Professional Concept Documentation	01 — Identity & Governance	June 15, 2026